

International Conference on Computational Science, ICCS 2017, 12-14 June 2017,
Zurich, Switzerland

Improving Performance of Multiclass Classification by Inducing Class Hierarchies

Daniel Silva-Palacios, Cèsar Ferri, and María José Ramírez-Quintana

DSIC, Universitat Politècnica de València,
Camí de Vera s/n, 46022, Valencia, Spain
dasilpa@posgrado.upv.es, {cferri,mramirez}@dsic.upv.es

Abstract

In the last decades, one issue that has received a lot of attention in classification problems is how to obtain better classifications. This problem becomes even more complicated when the number of classes is high. In this multiclass scenario, it is assumed that the class labels are independent of each other, and thus, most techniques and methods proposed to improve the performance of the classifiers rely on it. An alternative way to address the multiclass problem is to hierarchically distribute the classes in a collection of multiclass subproblems by reducing the number of classes involved in each local subproblem. In this paper, we propose a new method for inducing a class hierarchy from the confusion matrix of a multiclass classifier. Then, we use the class hierarchy to learn a tree-like hierarchy of classifiers for solving the original multiclass problem in a similar way as the top-down hierarchical classification approach does for working with hierarchical domains. We experimentally evaluate the proposal on a collection of multiclass datasets showing that, in general, the generated hierarchies not only outperforms the original (flat) classification but also hierarchical approaches based on other alternative ways of constructing the class hierarchy.

© 2017 The Authors. Published by Elsevier B.V.

Peer-review under responsibility of the scientific committee of the International Conference on Computational Science

Keywords: Multiclass classification, Class hierarchy inference, Hierarchy of classifiers

1 Introduction

In machine learning, classification is the problem of identifying to which of a set of categories (classes) a new instance belongs. When the categories contains more than two different labels, the problem is distinguished as multiclass classification. In the basic scenario of multiclass classification it is assumed that (1) only one class label is assigned to each instance (in other words, this is a single-class classification as opposite to multi-label classification that allows multiple class labels for each instance), and (2) the class labels are independent (that is, no relationship among the class labels as opposite to hierarchical classification that aims to classification problems where classes are organised into a hierarchical structure).

One of the main objectives when solving a classification task is to make predictions as “good” as possible, where the notion of “good” depends on the evaluation measure used to assess the quality of the classifier. For instance, in classification, accuracy is one of the most popular evaluation measure. However, obtaining good predictions is not always a simple task. And things are usually even more complicated in the multiclass case since here the classifier has to distinguish among a high number of classes to make the predictions. For this reason, in the last decades, numerous efforts have been done trying to improve classifier performance. Most approaches proposed in the literature fall into one of these three groups: a) to design better learning techniques, b) to apply some kind of transformation over the training data, and c) to modify or adjust the predictions given by the classifier. For instance, as an example of approach of type a), we can mention the notion of *ensemble learning*, a general term based on the idea of using more than one model to determine the predicted output, and that encompass a collection of techniques (such as bagging [3] or boosting [8], among others). The different approaches based on instance and feature selection ([17, 14]) and the oversampling and undersampling methods proposed to deal with imbalanced datasets ([15]) are examples of proposals that belong to group b). Finally, approaches based on threshold choice methods [12] or classifier calibration techniques [16, 21], that are applied to scoring models, are examples of approaches belonging to group c). All of these methods assume that the condition (2) aforementioned holds, that is, there does not exist any relationship between the class labels.

However, in multiclass classification, it has been shown that classification performance can also be improved by decomposing the multiclass problem into a hierarchy of intermediate classification problems that are smaller or less complex than the original one. Two alternative ways of decomposing the original problem have been explored. The first one relies on the idea that a problem becomes less complex if its dimensionality is reduced. For instance, in [22] the instance attributes are iteratively split into disjoint sets and then a new classification problem is defined for each partition. The second way relies on the idea that a multiclass problem becomes simpler if the number of classes is reduced. For instance, in [13] and [11] a class hierarchy is constructed (by assuming that there exists some relationship between the class labels) and then each internal node of the hierarchy defines a new classification problem involving only its children class labels. This latter way to address the multiclass problem is inspired in the top-down hierarchical classification method. The different approaches proposed mainly differ in the way in that the class hierarchy is automatically generated (based on instances or based on predictions) and/or which learning technique is used for learning the intermediate classifiers.

In this paper we also propose to decompose the original multiclass problem by using a class hierarchy from which a tree-like structure of classifiers is constructed. Similar to [11], the relationship between classes are derived from the confusion matrix of a “flat”¹ classifier (interpreting the confusion matrix as an indicator of how simple or hard for the classifier is to distinguish the classes). But instead of deriving the similarity between classes directly from the values of the confusion matrix (as [11] does), we transform the matrix trying to obtain as much information as possible from it. From the transformed matrix, we define a semi-metric function to compare the class labels that is used to generate the class hierarchy. We experimentally evaluate our proposal using a large collection of datasets and techniques.

The paper is organized as follows. In Section 2 we review some previous works and present our method for defining the similarity between classes from the confusion matrix that is then used for inducing the class hierarchy. In Section 3, we describe how to solve multiclass problems by using a class hierarchy. Section 4, presents the experiments we conducted in order to evaluate

¹In the literature, it is common to refer to a classifier that does not take into consideration any relationship between the class labels as “flat”.

our approach. Finally, Section 5 concludes the paper.

2 Inducing Class Hierarchies from Predictions

In this section we present our proposal for learning class hierarchies. First, we briefly summarise previous proposals for automatically inducing class hierarchies. Then, we define a new method for defining the similarity between classes that we use for obtaining the class hierarchy.

2.1 Related work

Most approaches for automatically generating class hierarchies are instance-based, that is, they build the hierarchy by applying a hierarchical clustering algorithm that makes use of any distance or similarity function between the instances. For example, some works about ontology construction [6, 2] and concept hierarchy learning from texts follow this approach.

Other instance-based approaches firstly pre-process or transform the instances and then the hierarchy is induced by using any distance function between the transformed items. For instance, [13] presents a method for automatically learn a hierarchy for document classification. The approach consists in applying a linear discriminant projection to transform the documents (represented as vectors) onto a low-dimensional space and then the similarity between two classes is defined as the distance between their centroids. A similar approach is presented in [19] for learning the ontologies used in a recommendation system developed using an item-based collaborative filtering technique. From the user-item matrix that describes the item ratings given by the users, the authors derive a collection of ontologies by using several distance functions and applying both agglomerative and divisive clustering.

Another alternative way of determining the similarity between classes is to use the predictions given by a classifier, especially constructed, in many cases, to this aim. In the framework of multi-label hierarchical classification, in [5] an ARTMAP neural network is used as a self-organizing expert system to derive hierarchical knowledge structures. A similar approach is presented in [4], where the authors use an association rule learner that extracts class hierarchies from the predictions given by a multi-label classifier. In the field of (non-hierarchical) multiclass classification, [11] presents an approach to improve document classification by combining Naive Bayes (NB) and Support Vector Machines (SVM). Concretely, once a NB classifier has been learnt, each row of its confusion matrix (after normalize it) is used to represent each class as a tuple of numerical values. Classes are compared by applying the Euclidean distance.

Our proposal, although also based on the predictions of a classifier, differs from the last approaches just mentioned in that the similarity between classes is not obtained either from the predictions of the classifier or applying any well-known distance function over the values of the confusion matrix. Instead of that, the confusion matrix is turned into a similarity matrix.

2.2 Calculating the similarity between classes

Let $C = \{c_1 \dots c_n\}$ be a set of n class labels and D be a set of labeled instances of the form $\langle x, y \rangle$ where x is a m -tuple whose components are the input attributes and $y \in C$ is the target attribute, i.e. the class. Given a flat classifier F trained using D , its confusion matrix M is an $n \times n$ matrix that describes the performance of F . The rows of M represent the instances in actual classes whereas the columns represent the instances in predicted classes. Thus, the elements of M placed at the main diagonal (M_{c_i, c_i}) are the instances of D that F correctly classifies whereas the rest of elements of M are the misclassification errors, that is $M_{c_i, c_j}, c_i \neq c_j$,

represents the instances of class c_i that F classifies as being of class c_j . For the sake of simplicity, in that follows we denote any class c_i by its subindex i .

In general, in any confusion matrix M we can observe the following facts: (1) misclassification errors are not usually uniformly distributed, which indicates that it is more difficult for the classifier to separate some classes than others; and (2) M is usually non-symmetrical, which means that to really measure the degree of confusion between two classes i and j we must take into account all the errors the classifier makes involving both classes, that is $M_{i,j}$ and $M_{j,i}$. Our proposal is based on this reasoning.

Given a confusion matrix M , the new function for comparing classes derived from M , we denote as d_S , is obtained by applying the following steps:

1. Normalisation: $\overline{M}_{ij} = \frac{M_{ij}}{\sum_{j=1}^n M_{ij}}$
2. Class overlap: the overlap degree between two classes i and j is

$$overlap(i, j) = \begin{cases} \frac{\overline{M}_{ij} + \overline{M}_{ji}}{2} & \text{if } i \neq j \\ 1 & \text{if } i = j \end{cases}$$

Applying this function we obtain a symmetric matrix $\overline{M}_O$ we call the *Overlapping Matrix*.

3. Similarity between classes: the similarity between two classes i and j is defined as

$$d_S(i, j) = 1 - overlap(i, j)$$

The similarity values belong to the interval $[0..1]$ where $d_S(i, j) = 0$ means that classes i and j are completely indistinguishables, whereas $d_S(i, j) = 1$ indicates that the classes are completely overlapping. After applying d_S we obtain another symmetric matrix $\overline{M}_S$ we call the *Similarity Matrix*.

Note that, (C, d_S) is a semi-metric space [10] since d_S satisfies the non-negativity, symmetry and identity conditions but, for some pairs of classes, d_S can violate the triangle inequality property (for instance, this is the case with classes a and b in Example 1).

Example 1 Let us consider a classification problem with four class labels $C = \{a, b, c, d\}$. Suppose that we have trained a flat classifier F (using any classification technique) whose confusion matrix M is depicted in Table 1a. From M we can see that F perfectly classifies the instances belonging to class d but makes a lot of mistakes classifying the instances belonging to the other classes. In fact, we could say that F is a poor classifier (whose accuracy is 0.55). Tables from 1b to 1d show step by step the different matrices obtained by applying our method. In this example it is easy to see that d_S does not hold the triangle inequality property since for classes a and b it holds $d_S(a, b) = 1 > d_S(a, c) + d_S(c, b) = 0.972$.

		Predicted			
		a	b	c	d
Real	a	20	0	30	0
	b	0	20	30	0
	c	30	30	10	0
	d	0	0	0	100

(a) Confusion Matrix M .

	a	b	c	d
a	1			
b	0.000	1		
c	0.514	0.514	1	
d	0.000	0.000	0.000	1

(c) Overlapping Matrix $\overline{M}_O$.

	a	b	c	d
a	0.400	0.000	0.600	0.000
b	0.000	0.400	0.600	0.000
c	0.429	0.429	0.143	0.000
d	0.000	0.000	0.000	1.000

(b) Normalised Confusion Matrix $\overline{M}$.

	a	b	c	d
a	0			
b	1.000	0		
c	0.486	0.486	0	
d	1.000	1.000	1.000	0

(d) The similarity Matrix $\overline{M}_S$.

Table 1: Example that illustrates the calculation of the similarity between classes from the confusion matrix of a flat classifier.

2.3 Inferring the class hierarchy

Once the distance matrix has been obtained, the class hierarchy is built by applying an agglomerative hierarchical clustering algorithm. The agglomerative approach works in a bottom-up manner starting by assigning each element to a single cluster (singleton) and then iteratively merging pairs of clusters until obtaining only one cluster. Clusters are joined based on the distance between them, referred as the linkage distance. Linkage distances are, among others, the complete distance (the maximum distance between elements of each cluster), the single linkage distance (the minimum distance between elements of each cluster) and the average linkage distance (the mean distance between elements of each cluster). The hierarchical clustering is graphically represented as a binary tree called dendrogram that shows not only how the clusters are grouped but the distance to which the grouping has been taking place. From the dendrogram, the hierarchy of classes is derived by considering that the clusters created between the leaves (the set of original classes) and the root constitute the internal nodes of the hierarchy. We could obtain different hierarchies depending on the linkage distance used.

3 Decomposing Multiclass Problems using a Class Hierarchy

Once the class hierarchy has been inferred, we propose to use it for solving the original multiclass problem by creating a tree of classifiers in the same way as the *Local Classifier per Parent Node (LCPN)* [20] does for hierarchical classification. LCPN is easy to implement and, hence, it is one of the approach most commonly used in the literature. This approach consists in training a flat classifier for every non-leaf node (or parent node) in the class hierarchy (including the root) to distinguish between its child nodes. As a resulting we obtain a set of classifiers arranged in a tree. Then, to classify a new instance the tree of classifiers is traversed in a top-down manner applying the classifiers from the root until a leaf is reached.

As the agglomerative clustering algorithm aggregates two groups at each step, the class hierarchy and the classifier tree derived from it are binary trees. That means that for a problem with n classes we have to train $(n - 1)$ flat classifiers. However, the number of flat classifiers we have to apply for classifying a new instance varies from $(n/2)$, when the class hierarchy (and

the tree of classifiers) is balanced, up to $(n - 1)$, when the class hierarchy is a chain. In that follows, we present a procedure to simplify the class hierarchy in order to reduce the number of internal nodes, and therefore the number of flat classifiers we have to train. This procedure, therefore, generally implies a complexity reduction of the method.

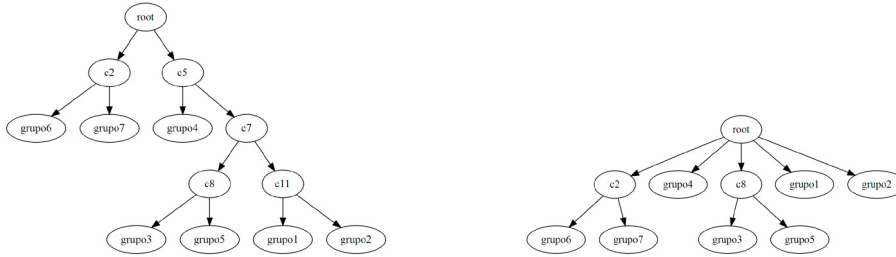


Figure 1: Example of the Hierarchy Compression Process applied to the Zoo Data Set. (Left) the class hierarchy given by the dendrogram. (Right) the compressed class hierarchy.

Algorithm 1 shows the compression method we propose to reduce the size of the class hierarchy. In order to do that, the class hierarchy is traversed in a depth-first search verifying that there is not two contiguous nodes that are at the same linkage distance; otherwise, the nodes are merged. In hierarchical clustering, the idea of using a post-process to reduce the dendrogram was also applied in [7], to develop a multidimensional hierarchical clustering.

Algorithm 1: Compression Algorithm

```

Function CompressionProcess (class_hierarchy) is
/* Depth-first search of the class hierarchy. */
node ← getRootNode(class_hierarchy);
node ← VerifyLevel(node);
children ← getChildren(node);
foreach child in children do
    node.child ← CompressionProcess(child);
return node;

Function VerifyLevel(node) is
/* While there is a child with same height. */
while (exists(node)) do
    children ← getChildren(node);
    foreach child in children do
        nodeHeight ← getHeight(node);
        childHeight ← getHeight(child);
        if nodeHeight == childHeight then
            node ← JoinNodes(node,child);
            break;
    return node ;

Function JoinNodes(nodeFather,nodeChild) is
nodeFather ← removeChild(nodeChild);
descendants ← getChildren(nodeChild);
foreach descendent in descendants do
    nodeFather ← addChildNode(descendent);
return nodeFather;

```

Figure 1 shows the compression process applied to the Zoo dataset (from the UCI repository [1]). As can be seen, the nodes *root*, *c5*, *c7* and *c11* in the hierarchy on the left are merged into one node that is the root of the resulting compressed hierarchy (depicted on the right). As a consequence, the number of flat classifiers to be learnt has been reduced from six to three. Note that, the node compression condition of being at the same linkage distance could be easily modified to allow merging nodes whose linkage distance does not exceed a certain threshold

depending on the domain or chosen by the user.

4 Experimental Analysis

In this section we evaluate the performance of our proposal to solve multiclass problems. For that reason we carried out a set of experiments following the method described in Section 3. The experiments were performed over 20 different datasets (Table 2) taken from the UCI [1] and the LIBSVM² public repositories, and the Reuters-21578 and Newsgroups datasets. All the datasets are multivariate, multiclass, non-hierarchical a priori, and the criterion we followed to select them was to include datasets of different size and different number of classes. We have preprocessed the datasets removing the instances with missing values. Additionally, we have applied under-sampling to datasets 11 and 15 (to cope with class imbalance) and an stratified sampling to datasets 19 and 20 (to reduce its size).

In order to analyse the suitability of the hierarchies generated by our method, we include two existing methods for generating hierarchies in the classes. First, a method that calculates the distance between the centroids of each class (we denote it as d_C) based on instances [13]. The second approach computes similarities by applying the Euclidean distance over the normalized confusion matrix generated by the flat classifier (we denote this distance as d_E) [11]. Our proposal, that uses a semi-metric function to compute the distance matrix, is denoted by d_S . We use the complete linkage distance in the agglomerative hierarchical clustering algorithm used to infer the class hierarchies (using d_S and d_E). In addition, to analyse whether the multiclass classification can be improved by using class hierarchies, we compare the results obtained using hierarchies with those get by the flat classifier, firstly trained for inducing the hierarchy.

DataSet	1	2	3	4	5	6	7	8	9	10
Id	Arrhythmia	Covtype	Dermatology	Flare	Forest	Glass	Letters	Nuclear	Pendigits	Reuters
NumInst	416	2100	358	1066	523	214	2600	525	7494	1000
NumAtt	330	54	34	19	27	9	16	80	16	58
NumClass	7	7	6	6	4	6	26	8	10	10
DataSet	11	12	13	14	15	16	17	18	19	20
Id	Satimage	Segmentation	Sports	TrafficLight	Usps	Vertebral	Yeast	Zoo	10News	20News
NumInst	1795	3000	8000	300	3000	620	1484	101	3700	6660
NumAtt	36	18	13	10	256	6	8	16	45	48
NumClass	6	7	10	6	10	4	10	7	10	20

Table 2: Information about the datasets used in the experiments: number of instances, attributes and classes.

In the experiments we apply eleven classification techniques in an R [18] script by means of the libraries caret [9], rpart, e1071 and C50. Specifically, we use the following classification algorithms: two decision trees “J48” and “C50”, a propositional rule learner “JRIP”, Naive Bayes “NB”, K-nearest neighbours “KNN”, a decision list “PART”, a recursive partitioning tree “RPART”, a neural network “NNET”, a parallel random forest “RF”, flexible discriminant analysis “FDA” and support vector machines “SVM”. We adopt a 10-fold cross-validation for the complete process and we use accuracy as the evaluation measure. In that follows, the letters S , C and E denote that the tree of classifiers has been built from a class hierarchy inferred from our semi-metric distance d_S , the distance between centroids d_C and the Euclidean distance d_E , respectively. Additionally, F stands for the flat classifier approach. Table 3 shows the results obtained. For each classification technique and dataset, the best result is underlined, and the

²[https://www.csie.ntu.edu.tw/~sim\\$cjlin/libsvmtools/datasets/](https://www.csie.ntu.edu.tw/~sim$cjlin/libsvmtools/datasets/)

best method using a hierarchy is highlighted in bold. As we can see, some algorithms were not able to process all datasets due to resource limitations.

In general, the results show that the methods that use a class hierarchy outperform, in terms of accuracy, the flat classification for most datasets and techniques except for the NB and SVM whose flat classifiers are mainly more accurate than the classifiers using hierarchies. Hence, we can say that, in general, the use of class hierarchies for decomposing the original multiclass problems seems to be suitable to address multiclass problems. Regarding the way in that the class hierarchies are induced, in average our semi-metric function d_S obtains the best results for almost all the techniques except for the NB where the distance between centroids d_C gives better accuracy values and C50 for which d_S and d_C obtain similar results.

Next, we analyse how the size of the datasets (in terms of number of classes and instances) affects methods performance. To do this, we have first grouped the datasets into three categories according to *NumClass*: small ($NumClass \leq 6$), medium ($6 < NumClass \leq 10$) and large ($10 < NumClass$). We observe that for the small group there are not big differences among using the flat, *S* and *C* approaches. For medium size datasets, flat and *S* approaches are the best for almost all the learning techniques and, finally, for large datasets, the classifiers that use our semi-metric distance achieve the best accuracy. Grouping the datasets according to the number of instances, we see that for small size datasets ($NumInst < 526$) all methods perform similarly; for medium size datasets ($527 < NumInst < 2101$) and large datasets ($NumInst > 2100$) the methods that use a class hierarchy are better than the flat ones, being the methods based on the Euclidean and the semi-metric distances the best ones.

5 Conclusions

This research proposes a method to improve accuracy on multiclass classification problems. The idea is that in situations where there exists a high number of classes, traditional methods find difficult to discern correctly the new observations given the high number of possibilities. The proposal consists in building specialised classifiers for the classes that present more common mistakes, i.e., to build a chain of specialised classifiers for simpler problems. Therefore, the method is based on the inference of the hierarchy of classes. From the confusion matrix obtained from train data we derive a similarity matrix, then an agglomerative clustering technique derives the hierarchy of classifiers. We propose a semi-metric to calculate the distances between classes given a confusion matrix. Also we introduce a method to compress hierarchies, which allow us to better represent the hierarchical structure and it also helps to reduce the complexity of hierarchical classification as it reduces the number of local classifiers.

Experiments with twenty multiclass datasets show the validity of our proposal. We show that the new technique is able to improve the accuracy with respect to the basic flat approach. We also include in the experiments other proposals to build the hierarchy of classifiers. The new method based on the semi-metric distance obtains a better performance for the majority of datasets.

As future work, we propose to analyze better the relation between number of classes and performance obtained by the chain of classifiers approach. We also are interested on studying the effect of class balance in the performance of the method. Finally, we plan to derive some ensemble methods based on the combination of the local classifiers of the hierarchy.

- [4] Florian Brucker, Fernando Benites, and Elena Sapozhnikova. Multi-label classification and extracting predicted class hierarchies. *Pattern Recognition*, 44(3):724–738, 2011.
- [5] Gail A Carpenter, Siegfried Martens, and Ogi J Ogas. Self-organizing information fusion and hierarchical knowledge discovery: a new framework using artmap neural networks. *Neural Networks*, 18(3):287–295, 2005.
- [6] Philipp Cimiano, Aleksander Pivk, Lars Schmidt-Thieme, and Steffen Staab. Learning taxonomic relations from heterogeneous sources of evidence. *Ontology Learning from Text: Methods, evaluation and applications*, 123:59–73, 2005.
- [7] Rakesh Dugad and Narendra Ahuja. Unsupervised multidimensional hierarchical clustering. In *Acoustics, Speech and Signal Processing, 1998. Proceedings of the 1998 IEEE International Conference on*, volume 5, pages 2761–2764. IEEE, 1998.
- [8] Yoav Freund and Robert E Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. In *European conference on computational learning theory*, pages 23–37. Springer, 1995.
- [9] Max Kuhn. Contributions from Jed Wing, Steve Weston, Andre Williams, Chris Keefer, Allan Engelhardt, Tony Cooper, Zachary Mayer, and the R Core Team. *caret: Classification and Regression Training*, 2014. R package version 6.0-30.
- [10] Fred Galvin and SD Shore. Distance functions and topologies. *The American Mathematical Monthly*, 98(7):620–623, 1991.
- [11] Shantanu Godbole, Sunita Sarawagi, and Soumen Chakrabarti. Scaling multi-class support vector machines using inter-class confusion. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 513–518. ACM, 2002.
- [12] José Hernández-Orallo, Peter Flach, and Cesar Ferri. A unified view of performance metrics: Translating threshold choice into expected classification loss. *Journal of Machine Learning Research*, 13(Oct):2813–2869, 2012.
- [13] T. Li, S. Zhu, and M. Ogihara. Hierarchical document classification using automatically generated hierarchy. *Intelligent Information Systems*, 29:211–230, 2007.
- [14] Huan Liu and Lei Yu. Toward integrating feature selection algorithms for classification and clustering. *IEEE Transactions on knowledge and data engineering*, 17(4):491–502, 2005.
- [15] V. López, A. Fernández, S. García, V. Palade, and F. Herrera. An insight into classification with imbalanced data: Empirical results and current trends on using data intrinsic characteristics. *Information Sciences*, 250:113–141, 2013.
- [16] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd international conference on Machine learning*, pages 625–632. ACM, 2005.
- [17] J Arturo Olvera-López, J Ariel Carrasco-Ochoa, J Francisco Martínez-Trinidad, and Josef Kittler. A review of instance selection methods. *Artificial Intelligence Review*, 34(2):133–143, 2010.
- [18] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2015.
- [19] V. Schickel-Zuber and B. Faltings. Using hierarchical clustering for learning the ontologies used in recommendation systems. In *13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD'2007*, pages 599–608, 2007.
- [20] Jr. Silla, N. Carlos, and Alex A. Freitas. A survey of hierarchical classification across different application domains. *Data Mining and Knowledge Discovery*, 22(1-2):31–72, 2011.
- [21] Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 694–699. ACM, 2002.
- [22] Blaž Zupan, Marko Bohanec, Janez Demšar, and Ivan Bratko. Learning by discovering concept hierarchies. *Artificial Intelligence*, 109(1):211–242, 1999.